Home SOC Lab: Simulating Attacks with Wazuh SIEM

Version 1.0

Nixon Rolle | April 2026

1. Project Summary

Home SOC Lab: Threat Detection with Wazuh & Suricata

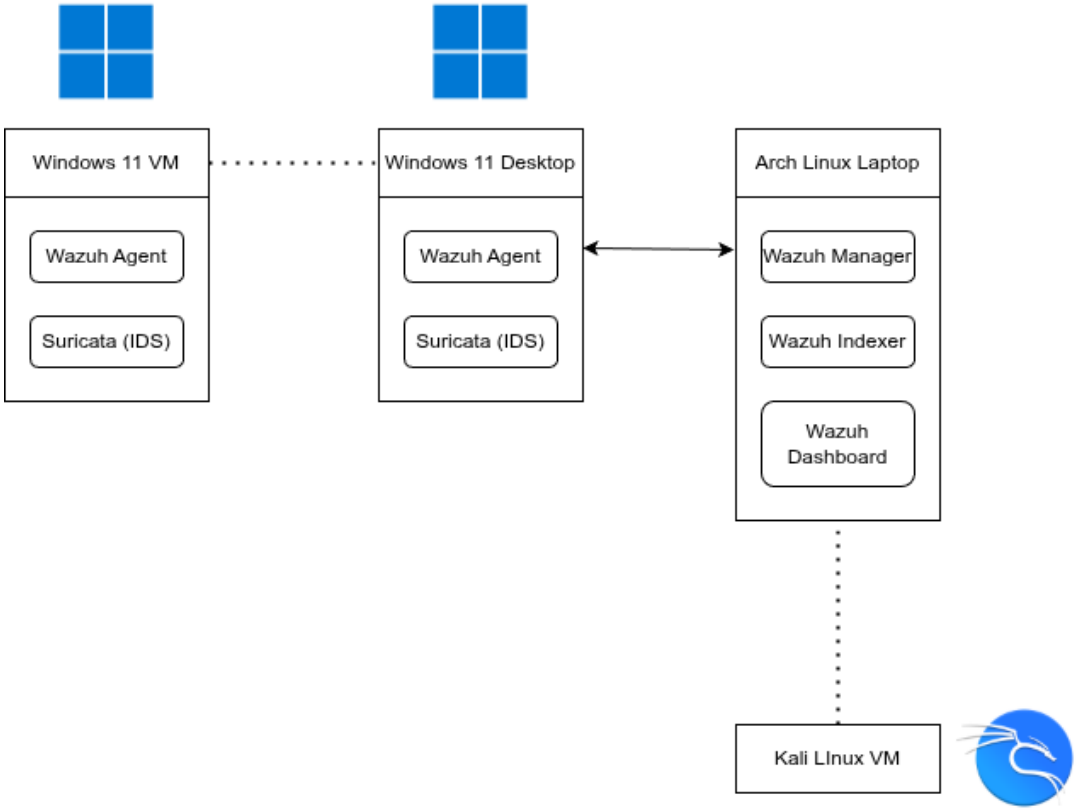
Built a Security Operations Center (SOC) laboratory to replicate and identify real-world cyberattacks using Wazuh SIEM and Suricata IDS. Executed comprehensive log collection, configured detection rules, and attack simulations encompassing network scans and brute-force attacks.

Technologies: Wazuh, Suricata, Kali Linux, Docker, Windows 11, Arch Linux

Key Skills Demonstrated

- SIEM (Wazuh)
- Intrusion Detection (Suricata)
- Log Analysis & Correlation
- Threat Detection & Rule Creation
- Network Traffic Analysis
- Docker & Linux
- Windows Security Logging

Lab Architecture



Data Flow:
Network traffic undergoes inspection by Suricata and is recorded in eve.json, which is subsequently gathered by the Wazuh agent. Logs from endpoints are transmitted to the Wazuh manager, where the events are examined and displayed on the dashboard.

Components:

Component	Technology	Purpose
SIEM	Wazuh (Docker)	Log collection, alerts, dashboard overview
Attack Machine	Kali Linux VM	Use of penetration tools to attack Windows endpoints.
IDS	Suricata	Network traffic inspection and attack detection
Primary Agent	Windows 11 Desktop	Forwards endpoint logs to Wazuh Manager

Vulnerable Agent	Windows 11 (VM)	Environment for advanced penetration testing and malware analysis
Log Source	Suricata eve.json	Logs network alerts, forwarded via Wazuh agent

Setup Summary

- The lab was deployed using Docker on Arch Linux for the Wazuh manager, with Windows-based agents forwarding logs. Suricata was configured to monitor network traffic and forward alerts via eve.json into the SIEM pipeline.

Achievements/Attack Simulations

ICMP Reconnaissance Detection

- Developed custom Suricata rules to detect ICMP echo requests (ping traffic)
- Validated real-time alert pipeline from network traffic to Wazuh dashboard
- Demonstrated ability to tune IDS rules from broad to specific detection

Detection Logic

```
alert icmp any any -> any any (msg:"ICMP Ping Detected"; itype:8;
sid:1000001; rev:1)
```

Detection Output


```
{"timestamp":"2026-04-12T17:02:22.334822-0400","flow_id":1494201739991346,"in_iface":"\\Device\\NPF_{B7924AFE-4078-479F-
8550-3C825C9AE4F4}","event_type":"alert","src_ip":"192.168.0.104","dest_ip":"192.168.0.100","proto":"ICMP","ip_v":4,"icm
p_type":8,"icmp_code":0,"pkt_src":"wire/pcap","alert":{"action":"allowed","gid":1,"signature_id":1000001,"rev":1,"signat
ure":"ICMP Ping Detected","category":"","severity":3},"direction":"to_server","flow":{"pkts_toserver":119,"pkts_toclient
":0,"bytes_toserver":11662,"bytes_toclient":0,"start":"2026-04-12T17:01:57.806647-0400","src_ip":"192.168.0.104","dest_i
p":"192.168.0.100"}}
```


Alert generated in eve.json

agent.name	rule.description	rule.level	rule.id
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601

Wazuh alert generated from Suricata ICMP rule

Nmap Network Scan Detection

- Integrated the Emerging Threats Open ruleset into Suricata for real-world threat detection
- Detected SYN and aggressive scans using signature-based IDS rules
- Validated visibility of reconnaissance activity within the SIEM pipeline

Detection Logic

```
alert tcp any any -> any any (msg:"ET SCAN Nmap"; flags:S;
threshold:type both, track by_src, count 20, seconds 3; sid:86601;
rev:1)
```

Detection Output

agent.name	rule.description	rule.level	rule.id
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601
WIN-01051FJOVJP	Suricata: Alert - ICMP Ping Detected	3	86601

Suricata alert triggered by Nmap SYN scan, matched against Emerging Threats rule (SID 86601) and forwarded to Wazuh

SSH Brute-Force Detection

- Implemented log-based detection using Wazuh by monitoring SSH authentication logs
- Developed custom rules to detect repeated failed login attempts
- Configured frequency-based alerting to identify brute-force behavior

Detection Logic:

```
<rule id="100200" level="5">
 <if_sid>5710</if_sid>
 <match>Failed password</match>
 <description>SSH failed login attempt</description>
```

```
</rule>
<rule id="100201" level="12" frequency="5" timeframe="60">
 <if_matched_sid>100200</if_matched_sid>
 <description>SSH brute-force attack detected</description>
</rule>
```

- These rules detect individual failed login attempts and escalate to a high-severity alert when multiple failures occur within a short time window.

Detection Output:

agent.name	rule.description	rule.level	rule.id
WIN-01051FJOVJP	SSH failed login attempt	5	100200
WIN-01051FJOVJP	SSH Brute force attack: 5+ failed logons in 60 seconds	12	100201
WIN-01051FJOVJP	SSH failed login attempt	5	100200
WIN-01051FJOVJP	SSH failed login attempt	5	100200
WIN-01051FJOVJP	SSH failed login attempt	5	100200
WIN-01051FJOVJP	SSH failed login attempt	5	100200

Wazuh alert triggered after multiple failed SSH login attempts within a 60-second window, indicating brute-force activity

Challenges & Solutions

The following challenges were encountered during implementation and resolved through systematic troubleshooting and configuration tuning.

1. Suricata `eve.json` Not Populating

Issue: Suricata was running but not generating alerts in `eve.json`
Cause: Misconfigured output (Redis enabled without a running instance) and missing/incompatible rule files
Resolution: Simplified configuration by removing Redis output, setting a valid log directory, and validating functionality using a minimal ICMP test rule

2. Wazuh Agent Connection Failure

Issue: Windows agent failed to connect to the Wazuh manager
Cause: Required ports (1514, 1515) were not exposed in Docker and Windows Firewall blocked traffic
Resolution: Exposed necessary ports in Docker configuration and created firewall rules to allow communication with the manager

3. JSON Decoder Overflow

Issue: Wazuh manager experienced log flooding and analysis service instability
Cause: Large Windows Security Event logs exceeded default JSON decoder limits
Resolution: Increased decoder limits and restricted log collection to relevant Event IDs (e.g., 4625) to reduce unnecessary data ingestion

4. Frequency Rule Not Triggering

Issue: Brute-force detection rule did not trigger despite repeated failed logins
Cause: Incorrect dependency on `same_source_ip` and inconsistent log field extraction
Resolution: Removed source IP dependency and refined rule logic to rely on event frequency within a defined timeframe

5. Empty Dashboard Results

Issue: Wazuh dashboard displayed no results despite active alerts
Cause: Incorrect time range selection and intermittent service communication issues
Resolution: Reset time filter and restarted Filebeat and Indexer services to restore data visibility

6. Docker Certificate Mount Errors

Issue: Wazuh manager failed to start due to certificate mount errors
Cause: Incorrect directory structure with invalid certificate files
Resolution: Regenerated certificates using the official script and redeployed the environment with a clean configuration

Impact

- Built a functional SOC lab capable of detecting real-world attack techniques
- Demonstrated end-to-end visibility from network activity to SIEM alerts
- Developed practical experience in detection engineering and log analysis

Planned Enhancements

- Implement active response to automatically block malicious IPs based on detection rules
- Integrate additional log sources (Linux systems, web servers, firewall logs) to simulate a multi-host environment
- Incorporate threat intelligence feeds to enhance detection of known malicious indicators
- Configure real-time alerting via email or Slack for high-severity events
- Expand detection coverage with automated vulnerability scanning integration